

Architecture and data warehousing notes

1. What we built

Our fictional company is moving off scattered spreadsheets onto one warehouse. HR and executives use a Streamlit web app to enter data and to explore dashboards. Behind it sit three MySQL databases, a Python object-oriented backend, and a set of stored procedures that move data between layers.

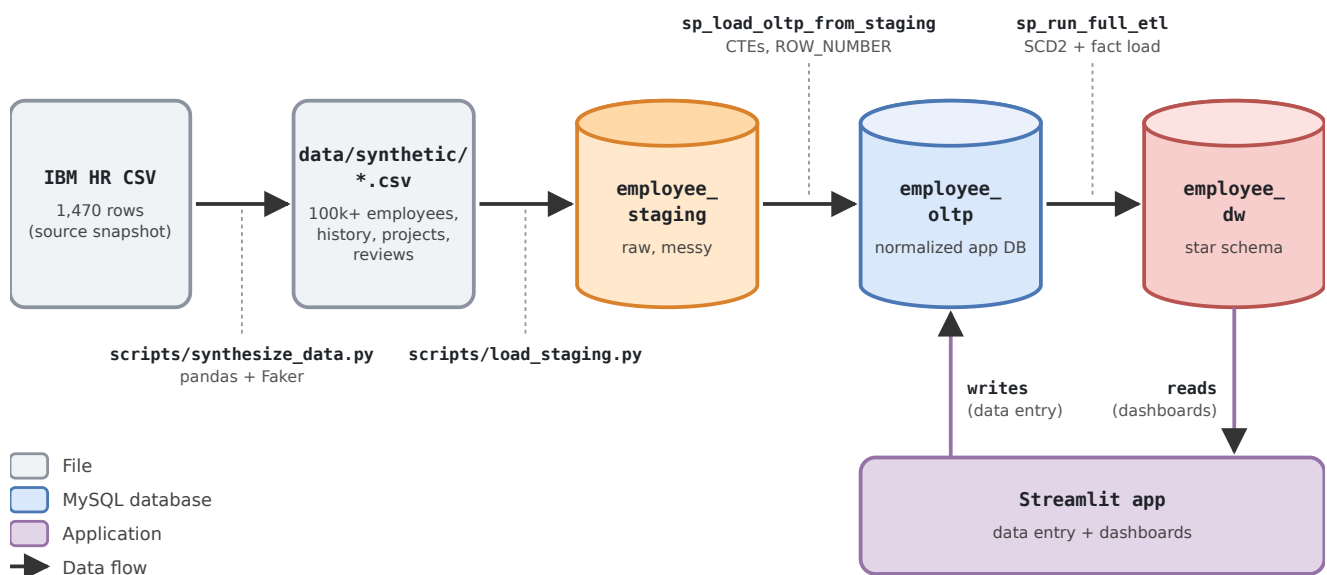


Figure 1 - End-to-end data flow

2. The data synthesizer

The IBM file is one snapshot, so we did three things to it:

1. **Scaled it.** Every original row is kept, then random rows are cloned until we hit 100,000 employees. Each clone gets slightly different age, income, commute and tenure, and a fresh name and email from Faker.
2. **Gave it a past.** About 30% of employees with two or more years of tenure get one or two earlier job states (a promotion, a department transfer, or a salary revision). We work backwards from today's state, so the history always ends in what the snapshot says is true now. This is the raw material for SCD Type 2.
3. **Added context.** 60 projects, around 120k assignments and five years of yearly performance reviews (about 387k rows).

We also left a little dirt in on purpose (duplicate employees and reviews, stray spaces, upper-case emails) so the cleaning step in SQL has real work to do.

3. OLTP design (`employee_oltp`)

Seven tables: `departments`, `job_roles`, `employees`, `employee_job_history`, `projects`, `project_assignments`, `performance_reviews`. Lookup values live once in their own tables; everything else points to them with foreign keys. `employee_job_history` stores one row per job version (`effective_start`, `effective_end` is NULL for the current one). Diagram: `docs/diagrams/oltp_er_diagram.mmd`.

4. OLAP design (`employee_dw`)

A star schema with `fact_performance_reviews` in the middle (one row per review) and four dimensions: `dim_employee`, `dim_department`, `dim_project`, `dim_date`. Diagram: `docs/diagrams/olap_star_schema.mmd`.

- **Surrogate keys.** Every dimension has an auto-increment key (`employee_key`, `department_key`, ...). The fact table only ever stores surrogate keys, never the OLTP ids. The OLTP ids are kept in the dimensions as business keys.
- **Unknown member.** `dim_project` has a row with key -1 ("No project") so reviews without a project don't need a NULL foreign key.
- **Smart date key.** `dim_date.date_key` is `yyyymmdd` (e.g. 20251205).

SCD Type 2 in `dim_employee`

Department, job role, job level and monthly income are tracked as history. When any of them changes we do **not** update the row. We close it and add a new one:

<code>employee_key</code>	<code>employee_id</code>	<code>department_name</code>	<code>start_date</code>	<code>end_date</code>	<code>is_current</code>
131071	101001	Sales	2025-03-01	2025-08-31	0
131072	101001	Human Resources	2025-09-01	9999-12-31	1

Because each review is linked to the employee version that was true *on the review date*, a review written while someone was in Sales stays in Sales forever, even after they move. Age, marital status, attrition and name are Type 1 (overwritten).

5. ETL

Staging to OLTP - `sp_load_oltp_from_staging` - `ROW_NUMBER() OVER (PARTITION BY employee_id ...)` keeps one row per employee. - The same trick on (`employee_id`, `review_date`) removes duplicate reviews. - `TRIM` / `LOWER` tidy names and emails; text is swapped for lookup ids. - It can be run more than once; rows already loaded are skipped.

OLTP to warehouse - `sp_run_full_etl` calls, in order: 1. `sp_load_dim_date` - builds the calendar from a digits cross join. 2. `sp_load_dim_department`, `sp_load_dim_project` - Type 1 update-then-insert. 3. `sp_load_dim_employee_scd2(employee_id | NULL)` - a CTE uses `LEAD()` to work out each version's end date, existing rows are closed or refreshed, unseen versions are

inserted with new surrogate keys. Pass one id to refresh one person. 4.

`sp_load_fact_performance_reviews` - loads only reviews not yet in the fact table, looking up the employee version by `review_date BETWEEN start_date AND end_date`.

6. Python backend (OOP)

Piece	Role
<code>DatabaseConnection</code>	Singleton that owns the connection pools (double-checked locking)
<code>DatabaseHandler</code>	Base class with query/transaction helpers; all managers inherit it
<code>Employee</code> , <code>Project</code> , <code>Review</code>	Entities with private attributes, validation and <code>to_row()</code>
<code>EmployeeManager</code> , <code>ProjectManager</code> , <code>ReviewManager</code>	CRUD against OLTP, inside transactions
<code>EtlManager</code>	Calls the warehouse stored procedures
<code>AnalyticsManager</code>	Read-only dashboard queries against the star schema

Business-rule problems raise `ValueError`; database problems raise `DatabaseError`. The UI turns the first into warnings and the second into errors.

7. How a department change flows end to end

1. HR submits the form in **Data entry > Change department**.
2. `EmployeeManager.change_department` opens one transaction: closes the current history row, inserts a new one, updates `employees`.
3. `EtlManager.refresh_employee` calls `sp_load_dim_employee_scd2(id)`.
4. The dimension now has the old version (closed) and the new one (current). The page shows both rows right away.

8. Dashboard

- Year-over-year trends by department (CTE + `LAG`).
- Top performers per department (CTE + `DENSE_RANK`).
- Attrition by department and overtime, plus a simple risk list for current staff.
- Project bottlenecks (`RANK` on average score and share of low ratings).
- An employee history viewer that makes the SCD2 rows visible.

9. Limits and honest notes

- The risk score is a rule of thumb, not a trained model.

- Because the data is synthetic, trends reflect how we generated it, not real behaviour.
- Retroactive edits to old job history rows aren't handled; changes are expected to be forward-dated.